Pràctica 1: Algorisme A^*

1. Plantejament del problema

L'objectiu d'aquesta pràctica, per a desenvolupar en dues sessions, és entendre i implementar l'algorisme A^* per cerca en grafs dirigits. L'algorisme haurà d'implementar-se del mode més general possible, però es provarà aplicant-lo a un joc anomenat Takuzu o Binairo. Aquest joc es pot formular com segueix:

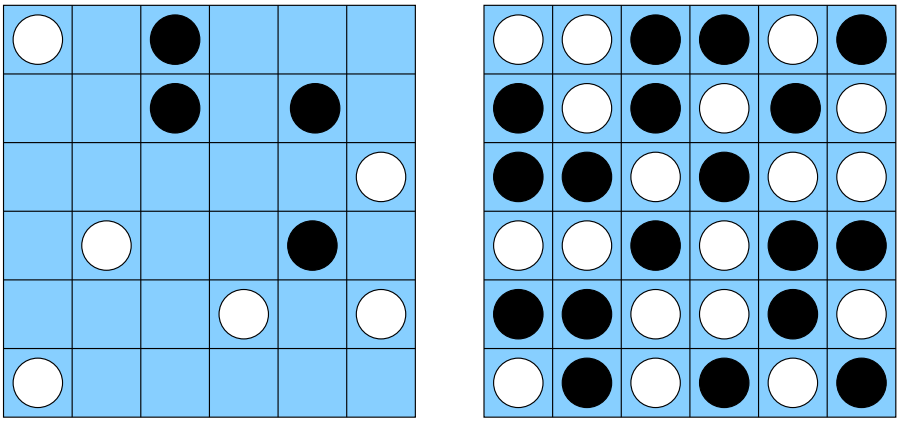
Siga un tauler quadrat de $n \times n$ caselles on n és un nombre parell. En cada casella del tauler pot haver-hi una fitxa blanca, una negra, o estar buida. Es parteix d'una configuració inicial donada amb algunes fitxes ja col·locades que compleixen les regles que detallarem a continuació, i l'únic jugador ha d'anar col·locant fitxes, de qualsevol color i en qualsevol casella a la seua elecció, de manera que **en tot moment** es complisca la següent regla:

> 1) No pot haver-hi més de dues caselles consecutives, ni en fila ni en columna, que tinguen fitxes del mateix color.

i a més la configuració final ha de complir aquestes regles:

> 2) Totes les caselles han d'estar ocupades.
> 3) En cada fila i en cada columna ha d'haver-hi el mateix nombre de caselles blanques i negres (que òbviament serà $\frac{n}{2}$).
> 4) Cap fila ni columna pot repetir-se.

Un exemple d'un Takuzu d'en 6×6 els seus estats inicial i final seria:



Note's que el jugador no pot situar una fitxa diguem de color 'c' on vulga, és a dir, no tots els moviments són legals: alguns violarien la regla 1, en situar tres fitxes de color 'c' consecutives. Uns altres violarien la regla 3, si ja hi ha $\frac{n}{2}$ fitxes de color 'c' en aqueixa fila/columna; i si falta una sola fitxa per a omplir una fila o columna, en completar-la podria també violar-se la regla 4.

Igualment, note's que situar una fitxa en posició legal **obliga** a situar unes altres: si la fitxa de color 'c' té al costat d'ella una altra del mateix color, qualsevol casella buida subsegüent haurà d'omplir-se amb una fitxa del color oposat. Similarment, si ja hi ha en una fila/columna $\frac{n}{2}$ fitxes de color 'c', totes les caselles buides d'aqueixa fila/columna hauran d'omplir-se amb el color oposat. A això ho cridarem **moviments obligatoris** i el programa els realitzarà automàticament.

En el nostre formalisme cada estat del tauler serà un **node** del graf d'estats. Els veïns d'aqueix node seran tots els estats que es generen en situar una fitxa en qualsevol de les posicions **legals** possibles **i completar amb tots els moviments obligatoris**. D'aquesta manera, l'estat veí a un donat en el qual hi havia p fitxes col·locades no necessàriament ha de tindre $p+1$ fitxes, sinó probablement algunes o moltes més.

El programa procedeix generant tots els nodes veïns de l'actual (en el sentit del paràgraf anterior), avaluant cadascun amb les funcions f i g el significat de les quals és el donat en classe per a l'algorisme

A^*, i aplicant aquest algorisme per a trobar el node objectiu, que és aquell en el qual tot el tauler està ple. La vostra tasca és programar l'algorisme A^* i provar-ho amb diverses funcions heurístiques $h(n)$. Sempre h haurà de tindre un valor menor quant millor (més desitjable) considereu l'estat del tauler al qual us porta cert moviment.

2. Implementació

2.1. Format de les dades

Es proporcionen alguns Takuzus per a proves, que es donen com a arxius de text, de grandàries progressivament majors. Els nomenats com tnne són de grandària nn i fàcils ('easy'); els nomenats tnnh són mes difícils ('hard').

En iniciar el joc el programa intern llegirà aquests arxius i els transformarà a la seua representació interna (un objecte de la classe `Board`, veure apèndix). A partir d'ací, heu de resoldre el problema usant el A^*, que programàreu com una funció, el qual generarà la solució que heu de mostrar en pantalla juntament amb el nombre d'iteracions usat.

3. Lliurament

S'entregarà en un arxiu comprimit:

- El codi realitzat PROU COMENTAT. Incloeu també el Makefile per a compilar-ho que ja es proporciona.
- Un arxiu de text LLEGIU-ME amb les instruccions per a compilar (generalment, simplement executar make), algun comentari de comparació dels resultats amb l'una o l'altra heurística i qualsevol comentari addicional que vulgueu fer.

ATENCIÓ: No es corregiran (i seran per tant invàlides) les pràctiques que ni tan sols compilen. Si compilen (en el sentit de generar un executable) però aquest no fa res que tinga sentit, la pràctica estarà probablement suspesa.

4. Apèndix: API de C++

En l'arxiu `takuzu.h` es defineixen:

- El tipus de dades definit per enumeració `Token={e,b,w}` per a indicar el possible contingut d'una casella (empty, black, white).
- El tipus de dades `Position` que és una estructura amb dos camps de tipus `unsigned` anomenats `row` i `col`, per a representar les coordenades d'una casella, que es compten des de 0.
- El tipus de dades `Move` que és una estructura amb dos camps, un de tipus `Position` anomenat `sq` i un altre de tipus `Token` anomenat `value`, per a representar el moviment consistent a posar una fitxa de color `value` en la casella `sq`.
- La classe `Board` per a representar un estat del tauler (un node) i en ella dues estructures de dades:
  `typedef list<Board> lb`
  per a representar una llista de taulers, és a dir, de nodes.
  `typedef list<pair<Move,Board> > lnei`
  per a representar una llista de nodes veïns a algun altre, on el primer element del parell representa el moviment que ens portarà a un cert estat, i el segon l'estat al qual arribem en realitzar efectivament aqueix moviment, **i els seus subseqüents moviments obligatoris**.

- La classe `Scores`, per a guardar els valors de les funcions `f` i `g` de qualsevol tauler.

Els constructors i funcions públiques (únics que podeu usar) de la classe `Board` són:

`Board(unsigned size)` Constructor que crea un tauler buit amb `size` caselles de costat. `size` ha de ser un nombre parell.

`Board(string file_name)` Constructor que crea un tauler a partir de l'arxiu de text `file_name`, en el format dels exemples que s'entreguen.

`Board(string file_name,ostream &outst)` Com l'anterior, però després de la càrrega de l'arxiu mostra per pantalla (si se li passa `cout`) o escriu en un arxiu de text obert (si se li passa el descriptor d'arxiu) l'estat inicial, i el mateix després de realitzar els moviments obligatoris.

`unsigned GetSize()` Funció que retorna el nombre de caselles de costat del tauler.

`Token Content(Position p)` Funció que retorna el contingut de la casella en la posició p.

`void Show(ostream &outst)` Procediment que mostra per pantalla (passant `cout`) o escriu en un arxiu de text (passant-li el descriptor de l'arxiu obert) el tauler en el seu estat actual.

`void GetNeighbours(lnei &listnei)` Procediment que ompli la variable `listnei` passada per referència amb una llista de parells Moviment/Tauler resultant per a tots els moviments legals que es poden fer des de l'estat actual. Com es va dir, el tauler de la llista resulta d'haver executat no sols el moviment, sinó tots els obligats per ell.

`bool Member(lb &listboards)` Funció que retorna `true` o `false` segons si el tauler actual està o no en la llista de taulers que es passa com a argument.

Per la seua banda, els constructors i funcions públiques (únics que podeu usar) de la classe `Scores` són:

`void SetfValue(Board b,float v)` Procediment que emmagatzema en la taula de la funció `f` el valor donat per al tauler que se li passa.

`float GetfValue(Board b)` Funció que retorna el valor actualment emmagatzemat de `f` per al tauler que se li passa, o $+\infty$ (en la pràctica, la constant `FLT_MAX`) si encara no s'ha fixat valor per a aqueix tauler.

`void SetgValue(Board b,float v)` Procediment que emmagatzema en la taula de la funció `g` el valor donat per al tauler que se li passa.

`float GetgValue(Board b)` Funció que retorna el valor actualment emmagatzemat de `g` per al tauler que se li passa, o $+\infty$ (en la pràctica, la constant `FLT_MAX`) si encara no s'ha fixat valor per a aqueix tauler.

Un programa típic de prova seria:

```
./takuzu_test t06h
```

que es cridaria des de la línia d'ordres passant com a argument l'arxiu d'entrada que conté el takuzu proposat.

Heu d'implementar la funció `bool AStar(...)` (penseu que paràmetres se li haurien de passar) que retornarà `true` o `false` segons si existeix o no solució. També heu d'implementar qualssevol funcions auxiliars que considereu necessàries Tot això pot anar simplement en l'arxiu principal, abans de `main`.

I finalment han d'implementar-se en l'arxiu `heuristic.cpp` les heurístiques, el prototip de les quals ha de ser en qualsevol cas el que s'indica en `heuristic.h`, és a dir:

```
float h(Board current_board,Move m,Board next_board)
```

Dins de la funció no és obligatori usar els seus tres paràmetres; depén de la vostra heurística. Per a escriure diverses heurístiques i provar-les successivament podeu escriure diferents arxius `heuristic.cpp`, o posar-les totes en aquest, usant directives condicionals de la forma

```
#ifdef nombre_HEURISTIC
(escriure la funció h..)
#endif
```

i usar el nom que li hajau donat a l'heurística com a valor de la variable `CHOICE` que apareix a l'inici del `Makefile`.